Local Setup

From a fresh clone to a running app with usable data, then the test suites.

If you only read one thing

Copy `.env.example` to `.env`, fill in `SESSION_SECRET`, and change nothing else. Everything else already points at the local Docker Postgres and at the stub auth and email providers, so no external service is needed.

1 Prerequisites

Tool	Version	Where it's pinned
Node	20+	Nowhere — README prose only. No `engines` field, no `.nvmrc`.
pnpm	9+	Implied by `pnpm-lock.yaml` (`lockfileVersion: 9.0`)
Docker	any recent	Optional — see §7
Postgres	17 via Docker, 13+ otherwise	`docker-compose.yml` pins `postgres:17-alpine`

Node and pnpm are **not** actually pinned in the repo. If a version-drift bug ever bites, that missing `engines` field is why.

2 The `.env`

```
cp .env.example .env
```

`.env.example` ships in a **deliberately non-booting state.** `SESSION_SECRET` is empty so that `z.string().min(32)` fails and the app refuses to start. That is the one line you must fill:

```
openssl rand -base64 48

SESSION_SECRET=<the 48-byte string from above>
```

Everything else in the example file already works as shipped:

Variable	Default	Why it's fine
`DATABASE_URL`	`postgres://van:van@localhost:5433/van_reservation`	Matches `docker-compose.yml` exactly
`TEST_DATABASE_URL`	`…/van_reservation_test`	Auto-created on first integration run
`AUTH_MODE`	`stub` — the example's value, **not** a schema fallback	No CGS credentials needed. The schema has no default, so deleting the line stops the app booting rather than quietly selecting the stub
`EMAIL_MODE`	`stub`	No AWS needed
`DISPATCH_SECRET`	blank	Optional; blank makes the sweeper 404 by design

`NODE_ENV` defaults to `development` when unset. Leave it alone locally.

> **Do not add `ROSTER_ALLOWLIST` or `ROSTER_ALLOW_ALL_WHEN_EMPTY`.** Older notes describe them as a sign-in gate. They do not exist in the schema and setting them does nothing. There is no eligibility allowlist for associates — any identity the provider resolves is admitted, and `admin_whitelist` only decides who additionally gets `admin_support`. See domain §2.

`DATABASE_URL` vs the `DB_*` parts

Two forms. `DATABASE_URL` set directly is what the example ships. Alternatively leave it blank and set `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD` and the app composes the URL — only when host, name, and user are all present.

DATABASE_URL always wins when set. The sharp edge:

> **dotenv never overrides a variable your shell already exports.** A `DATABASE_URL` inherited from your shell silently beats every `DB_*` value in `.env`. Before debugging a connection that points somewhere unexpected, run `echo "$DATABASE_URL"`.

3 First run

Run these in order. The order matters — see the note after the block.

```
pnpm install
pnpm db:up        # Postgres 17 on host port 5433
pnpm db:migrate   # all migrations
pnpm db:codegen   # regenerate src/modules/db/types.d.ts from the live schema
pnpm db:seed      # admin whitelist + client fleet roster
pnpm db:seed:dev  # sample reservations (dev only)
pnpm dev          # http://localhost:3000
```

Step	What it does	If it fails
`db:up`	Postgres on **5433**, healthchecked every 5s	Port 5433 already bound — usually a stale container or another local Postgres
`db:migrate`	Schema only, fast	On a database with existing data, the van/rental CHECK constraints validate against existing rows
`db:codegen`	Reads the live schema	Needs a reachable `DATABASE_URL`. With `DB_SCHEMA` set, needs `--default-schema` passed by hand (see §7)
`db:seed`	Upserts 9 whitelist rows and the client fleet (8 drivers, 4 vans)	Requires migrations first — it reads `roster_events`
`db:seed:dev`	Sample reservations and people	Refuses to run when `NODE_ENV=production`
`dev`	`next dev`	Empty/short `SESSION_SECRET`, or Postgres not up

> **`db:seed:dev` truncates `drivers` and `vans` before loading its own fixtures.** So after the documented order above you have the *dev* fleet — 5 drivers, 4 vans — not the client's 8-driver roster that `db:seed` just inserted. That is fine for local work, but it means the `fleet_roster` seed is effectively for production only. If you specifically need the real client fleet locally, run `db:seed` and skip `db:seed:dev`, accepting that you will then have no reservations to look at.

`db:seed` is production-safe and convergent — neither of its two seeds checks `NODE_ENV`, and both are designed to be re-run. `db:seed:dev` truncates and is gated against production.

4 Signing in

With `AUTH_MODE=stub`, **the password is ignored entirely.** The stub never reads it; the Domain ID alone selects a fixture identity. Type anything.

Two doors, and the door decides your role:

- `/login` — always gives you `associate`, even if you are whitelisted.
- `/admin/login` — requires `admin_support` and 403s otherwise.

So a whitelisted admin can deliberately sign in at `/login` to use the app as a requestor.

Start here

Domain ID	Who	Sign in at	You get
`AM65108`	Arielle Jimera	`/login`	A requestor session holding 5 of the 8 canonical reservations, plus roughly a sixth of the generated year
`AG80389`	Ruwi Joy Eribal	`/admin/login`	Admin, and a `super_admin` — the only role that sees `/roster`'s Admins tab
`AB12345`	Juan Dela Cruz	`/login`	A requestor with **no** bookings — the empty state

> `AM65108` is also an `ADMIN_WHITELIST_SEED` row with `super_admin: true`, and that does **not** leak into the `/login` session: `authenticate` sets `role = portal === "admin" ? "admin_support" : "associate"` (`service.ts:91`), and middleware gates every admin route on `role`. The session still carries `superAdmin: true`, but nothing reachable from the requestor side reads it. So this is a real associate view, and the same Domain ID at `/admin/login` is a real admin one.
>
> The other three canonical reservations belong to `Dizon, Marco`, `Reyes, Kaye` and `Salcedo, Liza`, and the generated dashboard rows draw from a six-person pool (`generated.ts:41-48`). None of those people are in `dev-identities.ts`, so nobody can sign in as them to see their own trips.

Every stub identity

Domain ID	Name	Role	Site	super_admin
`AG80389`	Ruwi Joy Eribal	admin_support	all	yes
`AM65108`	Arielle Jimera	admin_support	all	yes
`AM37315`	Norlen Denonong	admin_support	manila	no
`AL12138`	Zarra Crist Bartolo	admin_support	manila	no
`AH85664`	Sharon Gemma Lim	admin_support	manila	no
`AM03146`	Jezreel Mariz Gromia	admin_support	iloilo	no
`AL95338`	Ivy Balandra	admin_support	iloilo	no
`AH44229`	Angel Grace Mateo	admin_support	iloilo	no
`AJ40001`	Aishki Hyamero	admin_support	all	no — **test account, not staff**
`AB12345`	Juan Dela Cruz	associate	—	—
`CD67890`	Maria Santos	associate	—	—

Roles only take effect **after `pnpm db:seed`** has populated `admin_whitelist`. Before that, every Domain ID resolves to `associate` and `/admin/login` rejects everyone.

> Two Domain IDs that older docs suggest — `AJ29104` and `IB10001` — do not exist as identities and will fail to authenticate. `AJ29104` survives only as an embedded passenger *name* in seed fixtures.

What you can reach

Route gating lives in `src/middleware.ts`.

- **Admin-only**: `/dashboard`, `/master-list`, `/calendar`, `/reports`, `/drivers`, `/audit-logs`, `/roster`. A non-admin session is redirected to `/not-authorized?reason=role`.
- **Any session**: `/`, `/book`, `/manage`. Admins can visit these too — one person can be both a requestor and an admin.
- **No session**: redirected to `/login`, or `/admin/login` if the target was an admin path.

End-to-end check: sign in at `/admin/login` as `AG80389`, confirm `/master-list` and `/calendar` render seeded reservations, and confirm `/roster` shows an Admins tab. If reservations are missing, `db:seed:dev` did not run.

5 Tests

	pnpm test	pnpm test:int
Matches	`src/**/*.test.{ts,tsx}, excluding *.int.test.ts`	`src/**/*.int.test.ts`
Files today	81	24
Needs Postgres	no	yes
Parallelism	default	serial (`fileParallelism: false`)
Timeout	default	20s

Integration tests are self-bootstrapping. No manual `createdb`. The global setup connects to `TEST_DATABASE_URL`, and only if that fails does it open the `postgres` maintenance database and create the test database, then runs migrations once.

Before any of that it **refuses to run** if `TEST_DATABASE_URL` is unset, equals `DATABASE_URL`, or does not contain the substring `test` — a guard against wiping your dev database.

Per-test isolation is transaction rollback, not per-schema: `withRollback` runs the test inside a transaction and then unconditionally aborts it with a sentinel error. Tests leave no residue.

Per-schema isolation (`DB_SCHEMA`) is a *different* layer, for concurrent developers sharing one server — see §7.

`VITE_CONFIG_NATIVE_IGNORE_WARNING=true` is set inline in the test scripts. Nothing in this repo explains why; it appears to suppress a Vitest-internal warning about the native config loader.

6 Other commands

Command	Does	Run it
`pnpm lint`	`biome check`	Before every commit
`pnpm format`	`biome format --write`	To auto-fix
`pnpm typecheck`	`tsc --noEmit`	Before every commit
`pnpm db:migrate:make <name>`	Scaffolds a timestamped migration	For any schema change
`pnpm db:codegen`	Regenerates `types.d.ts` from the live schema	After every migration
`pnpm db:down`	Stops the container; the data volume survives	When you're done
`pnpm build / pnpm start`	Production build and serve	Before a deploy

7 Using a database you don’t own

Docker is optional; any Postgres 13+ works.

Leave `DATABASE_URL` and `TEST_DATABASE_URL` blank and set the parts instead: `DB_HOST`, `DB_PORT`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`.

DB_SCHEMA puts your tables in a named schema instead of `public`, applied as a libpq startup option (`?options=-c search_path=…`), so every unqualified query and migration resolves inside it. This is what lets integration tests run without `CREATE DATABASE` rights: with `DB_SCHEMA` set, tests use a `<schema>_test` sibling **schema** in the same database rather than a separate database.

DB_SSLMODE=no-verify for a private CA. `require` will fail against an untrusted certificate — `pg` treats `require` as verify-full.

Codegen needs the schema told separately, and the npm script does not pass it:

```
pnpm exec kysely-codegen --dialect postgres \
  --out-file ./src/modules/db/types.d.ts --default-schema <schema>
```

8 Things that will confuse you

An empty `SESSION_SECRET` fails twice, deliberately. `env.ts` requires 32+ characters, and `src/middleware.ts` carries its own independent length check. Without the second one, a too-short secret would verify JWTs fine at the edge while `env()` rejected it everywhere else — the app would boot green and every non-edge route would 500 on its first request.

A real `cgsauth` login against an unseeded database shows empty pages. `AUTH_MODE=cgsauth` admits any identity the CGS service authenticates, but every reservation, driver, and van on screen comes from the seeds. Correctly authenticated users, zero rows anywhere. Reads exactly like a broken read path; it isn't.

Mail in stub mode goes nowhere while reporting success. The stub queues, renders, and marks every notification `sent` in `notification_outbox`. The dispatcher reports success. Nothing is delivered. The only visibility is a dev console line:

```
[email:stub] → someone@example.com · cc: a, b · Subject (stub-3)
```

An environment that never sets `EMAIL_MODE=ses` looks healthy in every log and every table while delivering nothing. This is why `EMAIL_MODE` defaults to `stub` but the stub *refuses to construct in production*.

`/roster`'s Admins tab is invisible until `pnpm db:seed` runs. `super_admin` lives only on whitelist rows, so an unseeded database has no holders — and the tab is hidden even from the person the seed would have made one. Fail-closed by design, easy to mistake for a bug.

`/api/internal/dispatch-outbox` 404s by default. It needs `DISPATCH_SECRET` (16+ chars). You don't need it for local work — the post-response `after()` trigger still delivers. See operations.

See also

- Domain rules — the business rules
- Architecture — how the code is laid out
- Operations — environment, deploy, runbook
- Email and notifications — the notification pipeline and the templates